

Habilidades especiais

A mesma opção no menu, uma ação diferente para cada classe — e uma recarga que impede usar o golpe mais forte todo turno.

NESTA AULA

1. Criar um contrato comum para habilidades especiais.
2. Fazer Guerreiro e Mago sobrescreverem esse comportamento.
3. Reaproveitar a passagem dos turnos para controlar recarga.
4. Ligar tudo ao menu sem perguntar qual é a classe do jogador.

EXPLICAÇÃO E PRÁTICA INTERCALADAS

Cada explicação termina com duas questões práticas. O PDF mostra contratos, atributos e exemplos de outros domínios, mas não entrega os métodos do RPG prontos.

Onde vocês pararam

O jogo já possui classes diferentes. O Guerreiro não erra o ataque; o Mago recupera vida quando defende. Porém, o jogador ainda escolhe as mesmas três ações durante quase todo o jogo: atacar, defender ou abrir a mochila.

Ontem, o personagem aprendeu a guardar estado durante vários turnos. Um bônus temporário começa, permanece ativo e termina sozinho dentro de `iniciar_turno`. Hoje essa mesma noção de tempo será usada de outro jeito: uma habilidade forte entra em **recarga** e só volta a ficar disponível depois de alguns turnos.

ONTEM

Um contador dizia quanto tempo ainda faltava para remover um efeito.

HOJE

Um contador dirá quanto tempo ainda falta para permitir uma nova utilização.

A PERGUNTA CENTRAL

Como o menu pode oferecer apenas “Usar habilidade especial” sem conhecer Guerreiro, Mago ou qualquer classe que ainda será criada?

Arquivos de hoje

ARQUIVO

O QUE MUDA

<code>personagem.py</code>	Contrato comum e estado da recarga.
<code>classes.py</code>	Comportamentos especiais do Guerreiro e do Mago.
<code>main.py</code>	Uma nova opção no turno do jogador.
<code>itens.py</code>	Não precisa mudar.

01 Uma promessa comum

O menu chama a mesma operação para qualquer personagem.

02 Duas identidades

Guerreiro e Mago cumprem a promessa de maneiras diferentes.

03 Recarga

A habilidade fica indisponível por um número conhecido de turnos.

04 Jogo completo

Status, mensagens e uma terceira classe para provar o padrão.

EXPLICAÇÃO 1

Uma ação, muitas respostas

O combate já usa polimorfismo quando chama `atacar`. A linha é igual para todos, mas Guerreiro, Troll e Dragão podem executar comportamentos diferentes. Uma habilidade especial deve seguir a mesma forma.

O MENU SABE

Existe uma ação chamada habilidade especial, que recebe um alvo.

A CLASSE SABE

Qual é a habilidade, como calcula o resultado e qual mensagem apresenta.

O contrato

`habilidade_especial(self, alvo)` é a assinatura comum. Cada parte tem um papel:

PARTE	SIGNIFICADO	POR QUE EXISTE
<code>self</code>	O personagem que está agindo.	Permite acessar seus atributos e sua recarga.
<code>alvo</code>	Quem pode receber o efeito.	Mantém a chamada compatível com habilidades ofensivas.
<code>mesmo nome</code>	Uma promessa para todas as classes.	Evita decisões por tipo dentro do menu.

A classe `Personagem` oferece a versão geral. As classes filhas sobrescrevem o método. A versão geral não precisa adivinhar uma habilidade; ela apenas define o que acontece quando uma classe ainda não possui uma.

Exemplo parecido: tipos de notificação

```
class Notificacao:
    def enviar(self, destino):
        print("Nenhum canal foi configurado")
class Email(Notificacao):
    def enviar(self, destino):
        print("Enviando e-mail para", destino)
class SMS(Notificacao):
    def enviar(self, destino):
        print("Enviando SMS para", destino)
for aviso in [Email(), SMS()]:
    aviso.enviar("Ana")
```

O laço nunca pergunta se recebeu `Email` ou `SMS`. O objeto responde. No RPG, `turno_do_jogador` ocupa a posição desse laço.

Erro comum: não crie um `if` perguntando se o jogador é Guerreiro ou Mago. Isso obrigaria o menu a mudar toda vez que uma classe aparecesse.

PRÁTICA 1 · DUAS QUESTÕES

Abrir o caminho comum

Primeiro a chamada fica igual para todos. As diferenças entram depois.

QUESTÃO PRÁTICA 1

Definir a promessa em Personagem

Crie em `Personagem` o método `habilidade_especial(self, alvo)`. Nesta primeira versão, ele deve apenas informar que aquele personagem ainda não possui habilidade especial.

Explique por que o método pertence à classe-mãe mesmo que as versões interessantes sejam escritas nas filhas.

Teste isolado: chame o método diretamente em dois personagens. Neste momento, ambos ainda podem apresentar o comportamento geral.

QUESTÃO PRÁTICA 2

Uma nova opção no turno

Acrescente ao menu do jogador a opção “Usar habilidade especial”. Quando escolhida, ela deve chamar o método comum, passando o inimigo como alvo.

Não importe `Guerreiro` ou `Mago` dentro da função e não use `type`, `isinstance` ou comparação com o nome da classe.

Checkpoint: o menu reconhece a nova escolha e continua funcionando para qualquer objeto que seja um `Personagem`.

EXPLICAÇÃO 2

Classes com identidade própria

Sobrescrever não significa apenas trocar uma mensagem. A versão da filha deve expressar a identidade daquela classe. Para isso, a habilidade pode guardar números próprios e usar as regras que já existem no jogo.

Guerreiro: Golpe esmagador

O Guerreiro luta de perto. Sua habilidade pode causar um golpe físico mais forte que o ataque comum. O alvo ainda aplica a própria defesa por meio de `receber_dano`; a habilidade não deve alterar `_vida` de fora.

Um atributo como `dano_golpe` ou `bonus_golpe` deixa explícito o número que diferencia o golpe. Decida se ele representa o dano inteiro ou apenas uma parcela acrescentada ao ataque normal e mantenha essa interpretação em todo o método.

Mago: Bola de fogo

O Mago deve produzir um resultado diferente, não apenas outro nome para o mesmo golpe. A Bola de fogo pode causar dano mágico e ignorar a defesa. O Dragão já possui no código uma técnica que chega a um dano final mesmo passando por `receber_dano`; ele serve como pista, não como trecho para copiar sem entender.

Um atributo como `dano_magico` guarda a força própria da magia. O método continua recebendo `alvo`, exatamente como o do Guerreiro.

O QUE MUDA, O QUE FICA IGUAL

Mudam o cálculo, os atributos consultados e a mensagem. Permanecem o nome do método, os parâmetros e a chamada feita pelo menu.

Exemplo parecido: fretes diferentes

```
class Frete:
    def calcular(self, peso):
        return 0

class FreteComum(Frete):
    def calcular(self, peso):
        return peso * 2

class FreteExpresso(Frete):
    def calcular(self, peso):
        return 12 + peso * 3
```

As subclasses recebem o mesmo parâmetro, mas usam fórmulas próprias. Quem pede `frete.calcular(peso)` não precisa conhecer nenhuma delas.

PRÁTICA 2 • DUAS QUESTÕES

Duas respostas para a mesma chamada

Os valores são escolhas de design; o contrato não é.

QUESTÃO PRÁTICA 3

Golpe esmagador do Guerreiro

Escolha e guarde no Guerreiro um número que represente a força de seu golpe especial. Sobrescreva `habilidade_especial(self, alvo)` para causar um ataque físico mais forte e apresentar uma mensagem clara.

Use um método público do alvo para aplicar o resultado. Não toque diretamente em `alvo._vida`.

Teste: compare a mesma habilidade contra inimigos com defesas diferentes e explique por que o dano final muda.

QUESTÃO PRÁTICA 4

Bola de fogo do Mago

Escolha e guarde no Mago o dano de sua magia. Sobrescreva o mesmo método para que a Bola de fogo atravessasse a defesa do alvo sem acessar sua vida privada.

A assinatura precisa permanecer idêntica à do Guerreiro e à da classe-mãe.

Teste: use a magia contra dois inimigos com defesas diferentes. O resultado observado deve confirmar a regra que você escolheu para dano mágico.

EXPLICAÇÃO 3

Uma habilidade forte precisa esperar

Sem limite, a melhor decisão seria usar a habilidade especial todo turno. O menu teria quatro opções, mas uma delas tornaria as outras quase inúteis. A recarga cria uma escolha: usar agora e esperar, ou guardar para um momento mais difícil.

O estado da recarga

`_recarga_habilidade` representa quantos começos de turno ainda faltam para a habilidade voltar. Ele começa em zero: zero significa disponível. Um segundo valor, escolhido pela classe, representa o tamanho da espera iniciada depois de um uso bem-sucedido.

VALOR	LEITURA	O QUE A HABILIDADE DEVE FAZER
0	Disponível agora.	Executar e iniciar uma nova recarga.
maior que zero	Ainda faltam turnos.	Não executar e informar quanto falta.

O relógio já existe

`iniciar_turno` já atualiza defesa e efeitos temporários. A recarga também pertence ao personagem e também muda uma vez por turno. Portanto, o combate não ganha outro contador nem outra chamada.

Exemplo parecido: uma cafeteira em intervalo

```
class Cafeteira:
    def __init__(self):
        self._espera = 0

    def preparar(self):
        if self._espera > 0:
            print("A máquina ainda está aquecendo")
            return
        print("Café preparado")
        self._espera = 2

    def passar_minuto(self):
        if self._espera > 0:
            self._espera -= 1
```

A cafeteira verifica antes de agir, inicia a espera somente depois do sucesso e nunca deixa o contador ficar negativo. O personagem terá nomes e ações diferentes, mas enfrenta as mesmas decisões.

FALHAR NÃO DEVE GASTAR A AÇÃO ESCONDIDO

Se a habilidade estiver em recarga, decida o comportamento do turno. Para esta aula, a tentativa mostra a mensagem e encerra essa escolha; o jogador deverá esperar o próximo turno. Deixe essa regra visível no texto do jogo.

PRÁTICA 3 · DUAS QUESTÕES

Controlar a recarga

A recarga pertence ao personagem, mas cada classe pode escolher seu próprio prazo.

QUESTÃO PRÁTICA 5

Guardar e atualizar a espera

Prepare em `Personagem` o estado que representa a recarga atual, começando disponível. Depois amplie `iniciar_turno` para aproximar esse estado de zero uma vez por turno, sem produzir números negativos.

Preserve tudo que `iniciar_turno` já fazia com defesa e efeitos temporários.

Teste isolado: atribua temporariamente uma espera conhecida, avance os turnos e registre o contador depois de cada chamada.

QUESTÃO PRÁTICA 6

Proteger as duas habilidades

Faça cada habilidade verificar a disponibilidade antes de produzir seu efeito. Depois de um uso bem-sucedido, inicie a recarga escolhida para aquela classe. Durante a espera, mostre quantos turnos ainda faltam.

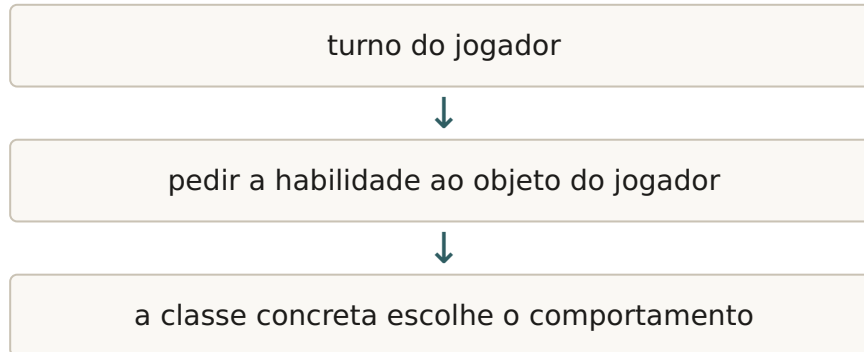
Não repita a contagem dentro das subclasses: elas decidem quando iniciar a espera; `iniciar_turno` é quem faz o tempo passar.

Teste: tente usar duas vezes seguidas, avance um turno por vez e identifique exatamente em qual turno a habilidade volta.

EXPLICAÇÃO 4

O menu continua sem conhecer classes

Depois das sobrescritas, a chamada criada na primeira prática não muda:



Se o objeto guarda um Guerreiro, acontece o Golpe esmagador. Se guarda um Mago, acontece a Bola de fogo. A variável não muda de nome, o menu não ganha um novo `if` e a chamada não recebe parâmetros diferentes.

A PROVA DO BOM DESENHO

Uma terceira classe jogável pode entrar alterando a lista de classes e o próprio `classes.py`. A função `turno_do_jogador` não precisa descobrir qual habilidade foi criada.

Informação para decidir

O jogador precisa saber se vale a pena escolher a ação. Acrescentar a recarga ao `mostrar_status` torna o estado interno observável sem expor o atributo para escrita. A mensagem pode mostrar “disponível” quando o contador vale zero e “faltam N turnos” nos demais casos.

Uma pequena matriz de testes

CENÁRIO	O QUE OBSERVAR
Guerreiro disponível	Executa seu golpe e inicia a própria recarga.
Mago disponível	Executa sua magia e inicia a própria recarga.
Qualquer classe esperando	Não aplica novamente o efeito e informa o prazo.
Contador chega a zero	A habilidade volta a funcionar sem ficar negativa.
Nova classe	O menu chama o novo comportamento sem alteração.

PRÁTICA 4 • DUAS QUESTÕES

Fechar e provar o padrão

A última dupla melhora a leitura do jogo e verifica se o menu está realmente aberto a novas classes.

QUESTÃO PRÁTICA 7

Mostrar a disponibilidade

Amplie `mostrar_status` para informar se a habilidade está disponível ou quantos turnos faltam. Não permita que código de fora altere a recarga diretamente.

Jogue com Guerreiro e Mago e confira se a informação muda no mesmo ritmo observado nos testes isolados.

Checkpoint: o jogador consegue decidir se deve escolher a opção especial antes de escolhê-la.

QUESTÃO PRÁTICA 8

Uma terceira habilidade

Transforme uma classe existente de sua escolha em opção jogável ou crie uma nova classe filha de `Personagem`. Dê a ela uma habilidade que não seja cópia exata das duas anteriores e uma recarga coerente com sua força.

Acrescente a classe em `CLASSES`, mas não altere a parte do menu que executa a habilidade.

Prova final: se a nova classe funciona com a chamada antiga, o polimorfismo está preservado.

Mesma escolha, heróis diferentes

Hoje o jogo ganhou uma ação nova, mas o ponto de POO foi manter essa ação estável enquanto cada classe decide o próprio comportamento.

- ☐ `Personagem` define o contrato de `habilidade_especial`.
- ☐ Guerreiro e Mago sobrescrevem o método com identidades diferentes.
- ☐ O menu chama a habilidade sem consultar o tipo do jogador.
- ☐ A recarga começa somente depois de um uso permitido.
- ☐ `iniciar_turno` atualiza a espera sem quebrar seus trabalhos anteriores.
- ☐ O status informa se a habilidade está disponível.
- ☐ Uma terceira classe entra sem criar outro desvio no menu.

O QUE FOI REUTILIZADO

Herança definiu a família, polimorfismo manteve a chamada comum, encapsulamento deixou o personagem cuidar da recarga e a passagem de turnos de ontem ganhou um segundo uso.